

ACESSÍLIA

Relatório de processo

Adaptação do material “Lógica matemática e conjuntos”: conversão de símbolos, semântica e os problemas encontrados no caminho

Baseado em execução real do pipeline

1. Do que trata este documento

Este relatório usa uma execução real do pipeline, o material “Lógica matemática e conjuntos”, para explicar como fiz pra converter símbolos de lógica e teoria dos conjuntos para português falado, e para contar a história dos problemas de semântica que apareceram nesse caminho: alguns já resolvidos antes e um que ainda está em aberto.

O material de entrada é uma página com: um parágrafo introdutório, uma tabela de cinco símbolos (pertence, não pertence, subconjunto próprio, subconjunto, conjunto vazio) e quatro exemplos, cada um com uma frase em português seguida da expressão matemática correspondente. É um material particularmente exigente para a camada semântica porque quase todo símbolo ali tem uma armadilha de leitura: “ $\in$ ” não é a letra “E”, “ $\subset$ ” não é uma seta, “ $\{1, 2, 3\}$ ” não é uma sequência de números soltos, é um conjunto.

1.1 O que o aluno recebeu, de fato

Antes de entrar no processamento, vale ver o resultado. Estas são linhas reais do TXT:

três pertence a abre chaves, um, dois, três, quatro,
fecha chaves

sete não pertence a abre chaves, um, dois, três, quatro,
fecha chaves

abre chaves, um, dois, fecha chaves está contido em abre
chaves,
um, dois, três, fecha chaves

naturais está contido ou é igual a inteiros

A última linha merece atenção: a expressão original era “ $\mathbb{N} \subseteq \mathbb{Z}$ ”. O sistema não leu “ene está contido ou é igual a zê”, ele reconheceu que $\mathbb{N}$ e $\mathbb{Z}$ são os conjuntos dos

números naturais e dos inteiros, e falou o nome por extenso. Essa leitura por significado (não por símbolo) é um tema que volta em vários pontos deste relatório.

2. O processamento, passo a passo

A seguir, os trechos principais do log desta execução, na ordem em que aparecem, com o que cada um significa.

2.1 Extração e planejamento

```
core.agents.agente_unico:executar:402 - Processando 1
pagina(s) para
logica_matematica_primeira_pagina.pdf com
estruturador=DoclingStructurer
core.agents.agente_unico:_catalogar:608 - Geometria
catalogada:
13 regioao(oes), 5 com spans
core.agents.agente_unico:_process_pdf_page:687 -
Extraidas 18 regioes
core.agents.planejador:planejar:472 - Plano:
9 text_clean, 4 titulo, 4 formula, 1 table
```

O que está acontecendo: o Docling extraiu 18 regiões da página (algumas se fundem depois, por isso o plano final trabalha com um número menor de blocos “vivos”). O planejador classificou o conteúdo em 9 trechos de texto limpo, 4 títulos, 4 fórmulas (os quatro exemplos numerados) e 1 tabela — a dos cinco símbolos. É essa classificação inicial que decide, região por região, se o conteúdo vai direto para a saída ou passa pelo especialista de visão.

2.2 Visão computacional: a tabela é reprovada na primeira tentativa

```
core.agents.agente_unico:_process_region_with_vision:1203
-
especialista Agno + critico: table
core.agents.critico_visual:....:675 - Critico REPROVOU
regiao table
(fiel=False, confianca=0.85): O símbolo '€' na imagem é
'€', que é
```

diferente do símbolo ' $\in$ ' mencionado na descrição.; O símbolo ' $\emptyset$ ' na imagem é ' $\emptyset$ ', mas o nome na tabela está escrito como 'Conjunto vazio' em vez de 'Conjunto vazio' com a grafia correta.

O que está acontecendo: este é o trecho mais interessante do log, e está detalhado na seção 4.3. Resumindo: o crítico visual comparou a descrição da tabela contra a imagem original e reprovou — com dois apontamentos. Note que o segundo apontamento compara “Conjunto vazio” com “Conjunto vazio”: as duas strings parecem idênticas no log. Isso é um comportamento conhecido do crítico (também observado em outros materiais): ele às vezes é rígido demais em rótulos matemáticos simples, e sinaliza divergência onde não há nenhuma visível no texto — o que pode ser um espaço, acento ou variante de largura que não aparece na leitura humana do log, mas ainda assim consome uma segunda tentativa. (Vou analisar mais a fundo depois)

2.3 Segunda tentativa: não converge

```
core.agents.critico_visual:...:719 - Regiao table nao
convergiu
apos 2 tentativas - marcando incerteza
```

O que está acontecendo: o especialista tentou descrever a tabela de novo, o crítico avaliou pela segunda vez, e a divergência não foi resolvida. Pela política do projeto, depois de duas tentativas sem convergência o sistema desiste de insistir e marca a região com o marcador interno “[verificação incerta]” — em vez de publicar uma descrição que não foi confirmada, ou travar o processamento esperando uma terceira tentativa.

2.4 As quatro fórmulas: aprovadas de primeira

```
core.agents.critico_visual:...:666 - Critico aprovou
regiao formula
(confianca=0.98) [repete 4 vezes — uma por exemplo]
```

O que está acontecendo: diferente da tabela, os quatro exemplos ($3 \in \{1,2,3,4\}$, $7 \notin \{1,2,3,4\}$, $\{1,2\} \subset \{1,2,3\}$, $\mathbb{N} \subseteq \mathbb{Z}$) foram aprovados de primeira, com confiança 0.98. Isso é coerente com o padrão observado em outros materiais: fórmulas isoladas e curtas raramente geram divergência; tabelas com texto ao redor do símbolo dão mais chance de ambiguidade entre “o que o símbolo é” e “o que o texto ao lado diz que ele é”.

2.5 O aviso sobre metadados ACESSÍLIA

```
core.agents.agente_unico:_registrar_acessilia:1466 -  
Metadado  
ACESSILIA nao registrado (No module named  
'core.services.acessilia_meta')  
[repete 5 vezes nesta execução]
```

O que está acontecendo: o mesmo aviso já visto em execuções anteriores; o módulo que grava a ficha de rastreabilidade de cada descrição não está presente nesta instalação. Aqui ele importa mais do que da última vez: é justamente esta execução que teve uma região reprovada e marcada como incerta (seção 2.3), e é exatamente para esse caso que a ficha de rastreabilidade serve: registrar CONFIANÇA, MOTIVO da incerteza e apontar a região para quem for revisar. Sem o módulo, a incerteza existe, mas não há nenhum rastro estruturado dela fora do log bruto. Eu tirei dessa versão, é uma ideia pra utilizar depois.

2.6 Editor textual: erro de schema

```
core.agents.editor_textual:revisar_documento:309 -  
Editor: JSON fora  
do esquema (2 validation errors for RelatorioEdicao  
inconsistencias.0.trecho — Field required  
inconsistencias.0.motivo — Field required); marcando  
necessidade  
de revisao humana
```

O que está acontecendo: o editor textual pede ao modelo de linguagem uma lista de inconsistências em formato estruturado (JSON com campos fixos: trecho, tipo,

motivo, sugestão). Desta vez o modelo devolveu um item com o campo tipo preenchido (“termo_suspeito”) mas sem trecho nem motivo, dois campos obrigatórios. A validação rejeitou a resposta, e por segurança o sistema marcou necessidade de revisão humana. Está detalhado, como item em aberto, na seção 4.4.

2.7 Exportação e áudio

```
core.services.coordenador_de_exportacao:gerar_artefatos:1
33 -
Artefatos gerados: docx, html, pdf, txt
renderers.sintetizador_de_voz:export_mp3:182 - Audio
(MP3) granular
exportado com sucesso (2 blocos)
```

O que está acontecendo: apesar dos dois avisos das seções 2.3 e 2.6, o pipeline seguiu até o fim e entregou os quatro formatos de documento mais o áudio — nada travou o processamento.

3. Os problemas de semântica, do mais antigo ao mais recente

Registro aqui os quatro problemas relacionados à conversão de símbolos e semântica que mais deram trabalho neste tipo de material — lógica e teoria dos conjuntos —, na ordem em que foram descobertos.

3.1 O mais antigo: fórmula tratada como texto, texto tratado como fórmula

Este é o problema fundacional, já coberto em detalhe num relatório anterior, mas relevante aqui porque foi ELE que causou o efeito mais visível já observado num material de lógica: um título de seção (“B = Operações entre conjuntos”) sendo processado como se fosse uma fórmula matemática, porque o pipeline antigo classificava o BLOCO INTEIRO como um tipo só.

A fala resultante saía como um produto de letras soltas: “b vezes o vezes p vezes e vezes r...”, porque um título maiúsculo em sequência (“Operações entre conjuntos”) parece, estruturalmente, uma cadeia de variáveis de uma letra multiplicadas — o mesmo padrão de “4ac”, só que sem serem números.

A correção de fundo foi a segmentação (módulo `pipeline/matematica/matematica_inline.py`), que resolve um bloco em texto + fórmula em vez de escolher um tipo só. Mas para este caso específico — texto que se PARECE com fórmula mesmo depois de isolado — foi criado um validador dedicado:

3.2 Símbolos de conjunto virando letra ou pontuação

Antes de existir uma tabela dedicada de operadores, os símbolos de lógica e conjuntos passavam por conversões genéricas demais e saíam errados de formas específicas:

- “U” (união) virando a letra “U” — o extrator via um caractere parecido com a letra maiúscula e não com o operador matemático;
- “\” (diferença de conjuntos) virando “barra invertida” genérica, como se fosse um caractere de formatação e não notação;

- “ $\forall$ ” (para todo) virando a letra “A” pela mesma confusão visual do primeiro caso.

A correção foi centralizar todos os operadores de lógica e conjunto — pertence, não pertence, subconjunto, subconjunto próprio, união, interseção, diferença, para todo, existe, existe único — numa tabela única,

`pipeline/matematica/registro_de_operadores.py`, com a forma LaTeX, o caractere Unicode, a fala em português e a precedência de cada operador NUM SÓ LUGAR. Antes de existir essa tabela, cada trecho do código (tokenizador, serializador de MathML, verbalizador de texto solto) tinha sua própria lista de símbolos, e as listas divergiam entre si.

A regra correspondente na validação é explícita e listada como bloqueio obrigatório: $\notin$ e $\nexists$ não são equivalentes; $\cup$ não pode virar “U”; $\backslash$ não pode virar “barra invertida”; $\forall$ não pode virar “A”. Qualquer substituição desse tipo bloqueia a publicação em vez de sair errada.

3.3 Variantes Unicode do mesmo símbolo — o achado deste log

O log desta execução (seção 2.2) trouxe um caso real: a imagem mostra “ $\in$ ” (U+2208, o símbolo padrão de pertencimento), mas o texto que o Docling extraiu do PDF trouxe “ ϵ ” (U+220A, uma variante mais estreita do mesmo símbolo, às vezes chamada de “small element of”). Visualmente quase idênticos; para o computador, dois caracteres diferentes.

Isso já tinha aparecido antes em outro material, e a camada de fala já tem a correção: o verbalizador de símbolos trata as duas variantes (e outras: “ $\ni$ ”, “ $\ni$ ”, “ $\ni$ ”, “ $\ni$ ”) como sinônimos para fins de LEITURA — “ $\in$ ” fala “pertence a” exatamente como “ ϵ ” falaria. Essa parte está resolvida e é por isso que o TXT deste pacote saiu correto (“três pertence a...”, seção 1.1).

O que este log mostra é a outra metade do problema, que é diferente e não tem uma correção de código simples: a CÉLULA VISUAL da tabela no HTML mostra o glifo exatamente como foi extraído — “€”, não “€” — porque para quem enxerga a tela, a coluna “Símbolo” existe para ser vista, não só ouvida, e reescrever o caractere que a imagem realmente mostra seria inventar informação. O crítico visual capturou exatamente essa divergência entre o que a imagem tem e o que o texto extraído registrou, e reprovou por isso — o comportamento correto: a falha está na extração (o glifo errado veio do PDF), e o crítico é a camada que existe para pegar esse tipo de erro antes que ele chegue ao aluno.

Fica como observação, não como bug fechado: se esse tipo de variante aparecer com frequência, vale ensinar o extrator (ou uma etapa de normalização logo depois dele) a preferir a forma padrão U+2208 quando a fonte do PDF permitir — hoje a correção só existe no lado da fala, não na extração.

3.4 O editor textual não tolera schema incompleto

Este é um problema real, ainda não corrigido, e que este mesmo log expôs (seção 2.6). O editor textual usa um schema Pydantic ([RelatorioEdicao](#)) para validar a resposta do modelo de linguagem, e rejeita qualquer inconsistência que não venha com TODOS os campos obrigatórios preenchidos (trecho, tipo, motivo). Desta vez o modelo devolveu um item incompleto e a resposta inteira foi descartada.

O editor textual ainda não tem muita tolerância: um campo faltando invalida o relatório inteiro, mesmo que os outros campos estivessem certos. A recomendação é aplicar o mesmo padrão de tolerância aqui — extrair o que der para validar, e só marcar para revisão humana a PARTE que realmente veio incompleta, não o relatório inteiro. Fica registrado como item de trabalho futuro.

4. Como cada símbolo deste material é tratado, na prática

A tabela a seguir mostra, para cada símbolo que aparece no material de lógica e conjuntos, o que o código faz: o nó que ele vira na árvore, e a fala que sai no TXT e no áudio.

Símbolo	Nó / tratamento	Fala em português
$\in$	Membership	pertence a
$\notin$	NonMembership	não pertence a
$\subset$	(subconjunto próprio)	está contido em
$\subseteq$	(subconjunto)	está contido ou é igual a
$\emptyset$	(conjunto vazio)	conjunto vazio
$\{ \quad , \quad , \quad \}$	ConjuntoLiteral	abre chaves, ..., fecha chaves
$\mathbb{N}$	Symbol (com papel semântico)	naturais
$\mathbb{Z}$	Symbol (com papel semântico)	inteiros
$\cup$	Union	união
$\cap$	Intersection	interseção
$\setminus$	SetDifference	diferença de conjuntos
$\forall$	ForAll / Quantificador	para todo
$\exists$	Exists / Quantificador	existe
$\exists !$	Quantificador (espécie única)	existe um único

5. Resultado final desta execução

O material foi processado e os cinco formatos de saída foram entregues. Três das quatro fórmulas (e agora, com a devida ressalva, a leitura da tabela) preservam o significado matemático correto: pertencimento, inclusão própria e não própria, e

leitura semântica dos conjuntos numéricos ($\mathbb{N}$ como “naturais”, não como a letra “ene”).

Como conclusão geral: o padrão de acerto deste material confirma que os símbolos de lógica e conjuntos — que têm fama de serem os mais fáceis de confundir visualmente ($\in$ com E, $\subset$ com uma seta, $\cup$ com U) — hoje passam pela mesma árvore única e pelos mesmos validadores das fórmulas de álgebra e física já testadas em relatórios anteriores. O que ainda falha, falha em pontos específicos e já mapeados: a fronteira entre extração de imagem e semântica (seção 4.3), e a tolerância de um agente específico a resposta incompleta do modelo de linguagem (seção 4.4) — não mais na conversão de símbolo em si.